



REDDIT CLONE

CSE343: Web Development

Contents

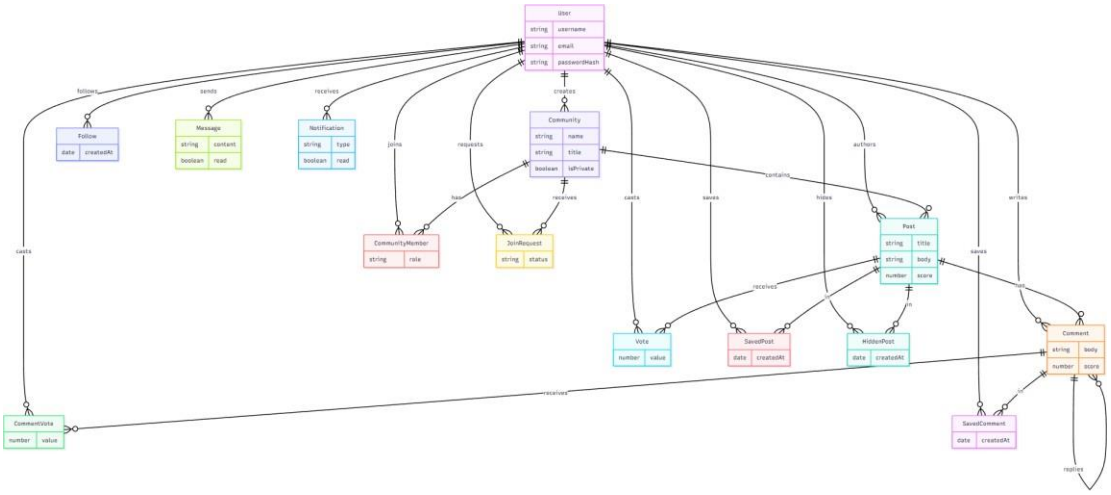
1. Introduction....2
2. Technology Stack....3
2.1 Frontend Technologies....3
2.2 Backend Technologies....3
2.3 Database Platform....3
3. System Architecture....3
4. Database Design Overview....3
4.1 Database Design Principles....4
4.2 Core Collections....4
5. Entity Relationships....4
6. Example Database Query Patterns....5
6.1 Retrieving the Newest Posts in a Community....5
6.2 Retrieving Top Posts in a Community....5
6.3 Retrieving Root-Level Comments....5
6.4 Retrieving Saved Posts for a User....5
6.5 Checking Community Membership....5
6.6 Retrieving Unread Notifications....5
7. Design Considerations....6
7.1 Choice of MongoDB....6
7.2 Denormalization Strategy....6
7.3 Relationship Modeling....6
7.4 Indexing....6
7.5 Full-Text Search....6
8. Testing the requirements....6
8.1 Login and signup page....6
8.2 Viewing/Editing User Profile....7
8.3 Creating a community....8
8.4 Joining/Leaving a community....9
8.5 Creating a post in a community....10
8.6 Viewing posts in a community....11
8.7 Viewing feed page....11
8.8 Upvoting/Downvoting posts....12
8.9 Commenting on posts....13
8.10 Searching for communities and users....13
9. Validation....14
9.1 Schema-Level Validations....14

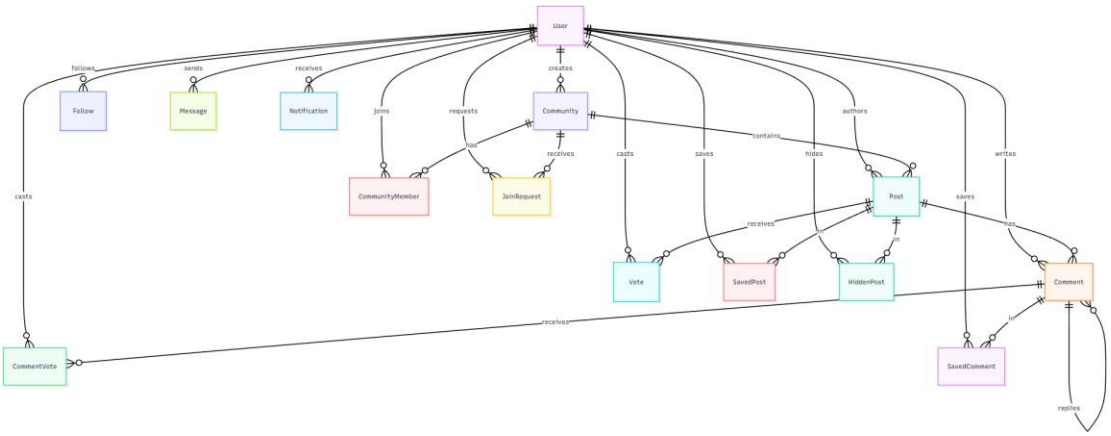
9.2 Route-Level Validations 15
9.3 Rate Limiting Validations 16
9.4 Authentication and Authorization Validation 16
10. API docx 16
11. Github and Deployment Link 17
12. Team Contributions 17

1. Introduction

Reddit is one of the most popular online discussion platforms, built around user-created communities where people share content and interact through posts, comments, and voting. Each community focuses on a specific topic, allowing users to engage in discussions that match their interests.

This project recreates the core functionality of Reddit as a web application. Users can create accounts, customize their profiles, join or create communities, and publish posts that other users can interact with. The platform supports commenting and voting, allowing discussions to grow organically while highlighting popular content. Users can also explore posts from different communities through a centralized feed and search for communities or users.





2. Technology Stack

2.1 Frontend Technologies

The frontend is responsible for user interaction and presentation. It was implemented using:

- React.js for component-based UI development
- React Router for client-side routing
- Axios for HTTP communication with the backend
- Tailwind for layout and styling

2.2 Backend Technologies

The backend handles application logic, authentication, and database communication:

- Node.js as the runtime environment
- Express.js as the web framework
- MongoDB as the primary database
- Mongoose as the Object Data Modeling (ODM) library

2.3 Database Platform

The database is hosted using MongoDB Atlas, providing a cloud-based, scalable data storage solution.

3. System Architecture

The application follows a client–server architecture, consisting of three primary layers:

1. Presentation Layer (Frontend):
   Handles user interaction and renders data received from the backend.
2. Application Layer (Backend):
   Exposes RESTful APIs, processes requests, enforces business logic, and communicates with the database.
3. Data Layer (Database):
   Stores persistent data using MongoDB collections and Mongoose schemas.
4. External Services**:** Cloudinary (media storage), Groq AI (content summarization)

This layered architecture improves maintainability, scalability, and separation of concerns.

4. Database Design Overview

The database design is implemented using MongoDB, a document-oriented NoSQL database. Each major entity in the system is represented as a separate collection, with relationships established using ObjectId references.

4.1 Database Design Principles

- Avoid deeply nested documents for scalability
- Use reference-based relationships for many-to-many associations
- Apply denormalization for frequently accessed computed values
- Define indexes to optimize query performance

4.2 Core Collections

The primary collections in the system include:

- User
- Community
- CommunityMember
- JoinRequest
- Post
- Comment
- Vote
- CommentVote
- SavedPost
- SavedComment
- HiddenPost
- Follow
- Message
- Notification

5. Entity Relationships

- A User can create multiple Posts and Comments
- A Community contains many Posts
- Users join communities through CommunityMember
- Votes are stored separately using Vote and CommentVote
- Users may follow other users
- Users can exchange private messages
- Notifications track system events and user interactions
- Private communities use JoinRequest for membership approval
- Comments support nested replies via self-referencing (parent field)

- Users can save posts/comments and hide posts

This approach allows the system to scale efficiently as the number of users and interactions increases.

6. Example Database Query Patterns

6.1 Retrieving the Newest Posts in a Community

The following query retrieves the most recent posts in a specific community:

```
Post.find({ community: communityId })
  .sort({ createdAt: -1 })
  .limit(25);
```

6.2 Retrieving Top Posts in a Community

This query retrieves posts ordered by score, representing community popularity:

```
Post.find({ community: communityId })
  .sort({ score: -1 })
  .limit(25);
```

6.3 Retrieving Root-Level Comments

This query retrieves only top-level comments for a post:

```
Comment.find({ post: postId, parent: null })
    .sort({ createdAt: 1 });
```

6.4 Retrieving Saved Posts for a User

This query retrieves all posts saved by a user:

```
SavedPost.find({ user: userId }).populate("post");
```

6.5 Checking Community Membership

This query determines whether a user is a member of a specific community:

```
CommunityMember.findOne({
  user: userId,
  community: communityId
});
```

6.6 Retrieving Unread Notifications

This query retrieves all unread notifications for a user:

```
Notification.find({ user: userId, read: false })
      .sort({ createdAt: -1 });
```

7. Design Considerations

7.1 Choice of MongoDB

MongoDB was selected due to its flexibility, scalability, and suitability for handling large volumes of user-generated content with evolving schemas.

7.2 Denormalization Strategy

Certain fields such as post scores, comment counts, and community member counts are denormalized to reduce query complexity and improve read performance.

7.3 Relationship Modeling

Many-to-many relationships are implemented using dedicated collections rather than embedded arrays, ensuring scalability and efficient querying.

7.4 Indexing

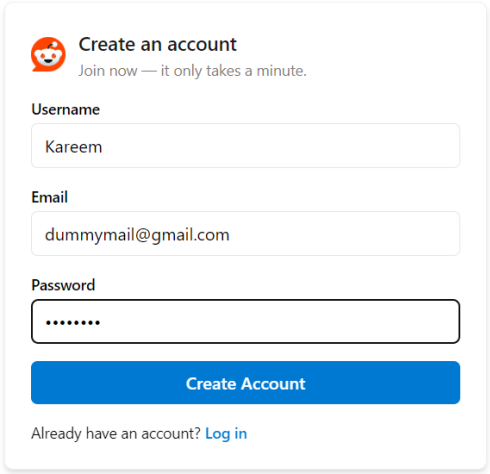
Indexes are applied to frequently queried fields such as usernames, emails, community feeds, and vote relationships to optimize performance.

7.5 Full-Text Search

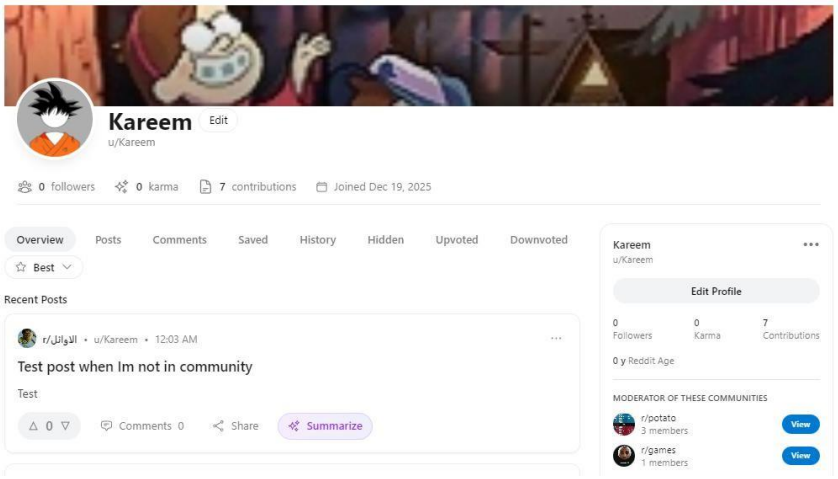
Text indexes are applied to post titles and body content to enable efficient keyword-based search functionality.

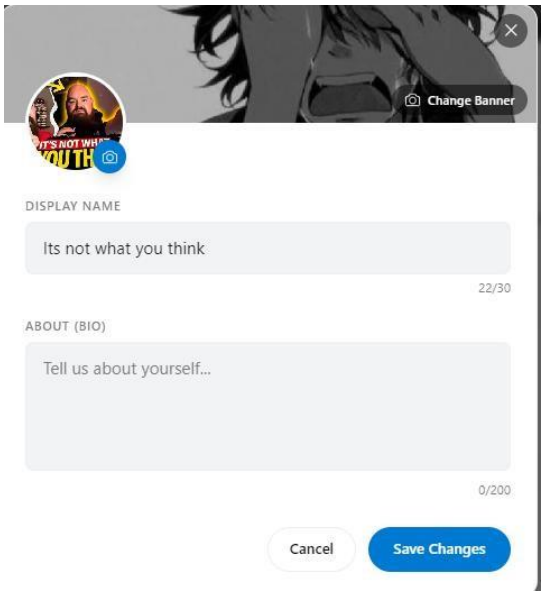
8. Testing the requirements

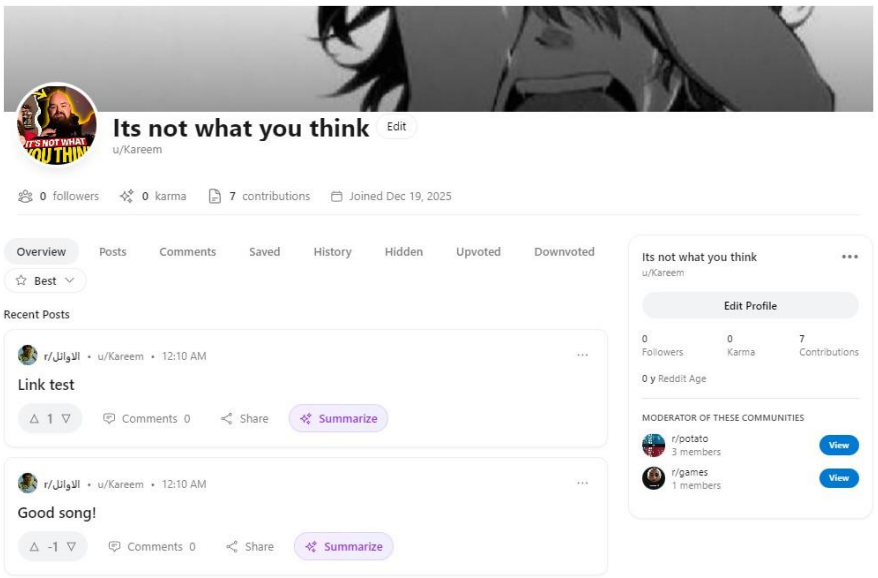
8.1 Login and signup page



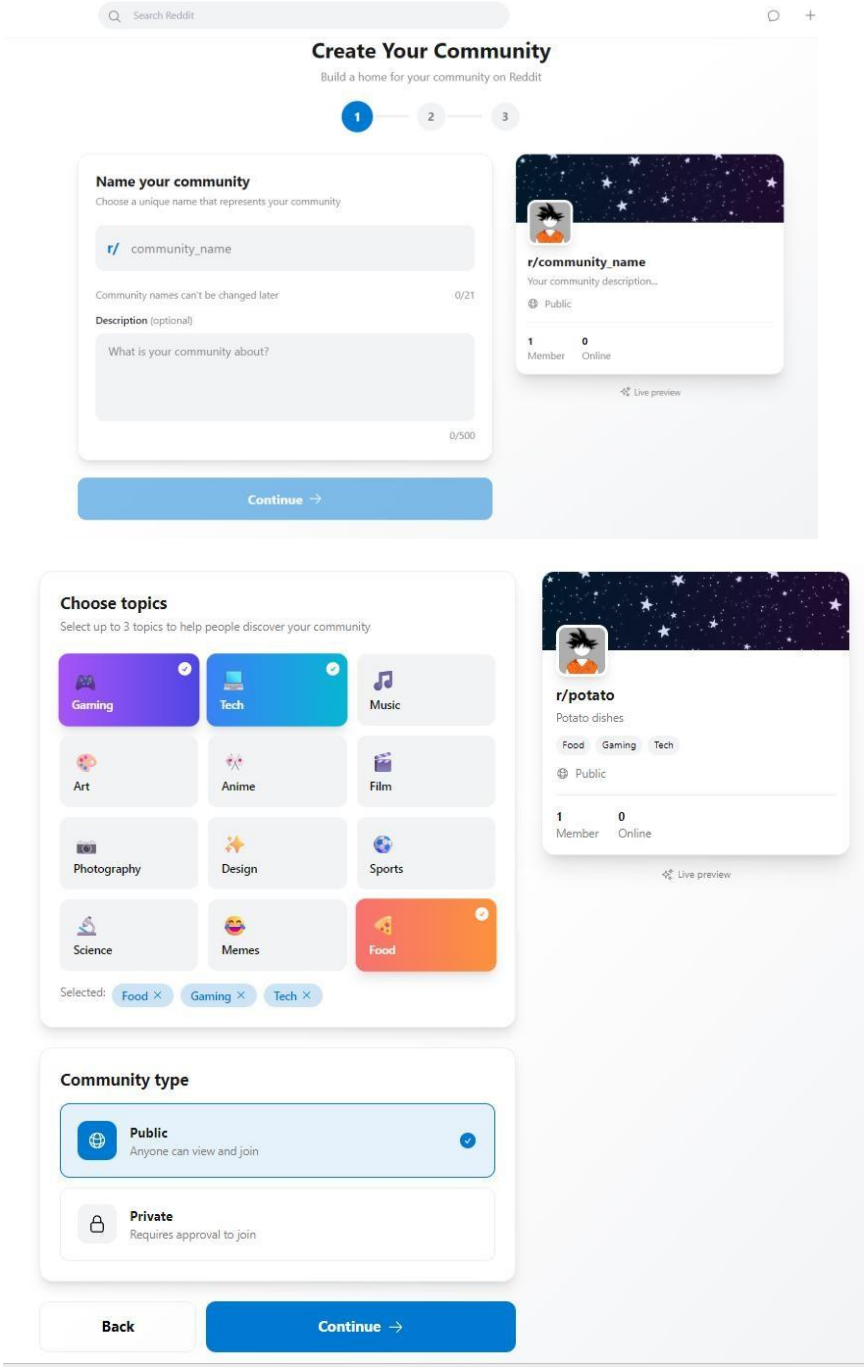
8.2 Viewing/Editing User Profile

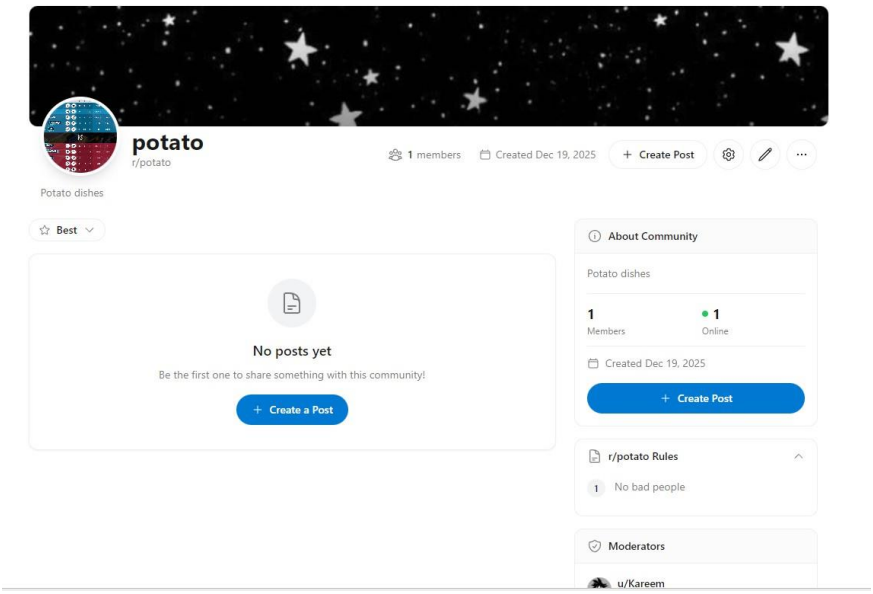




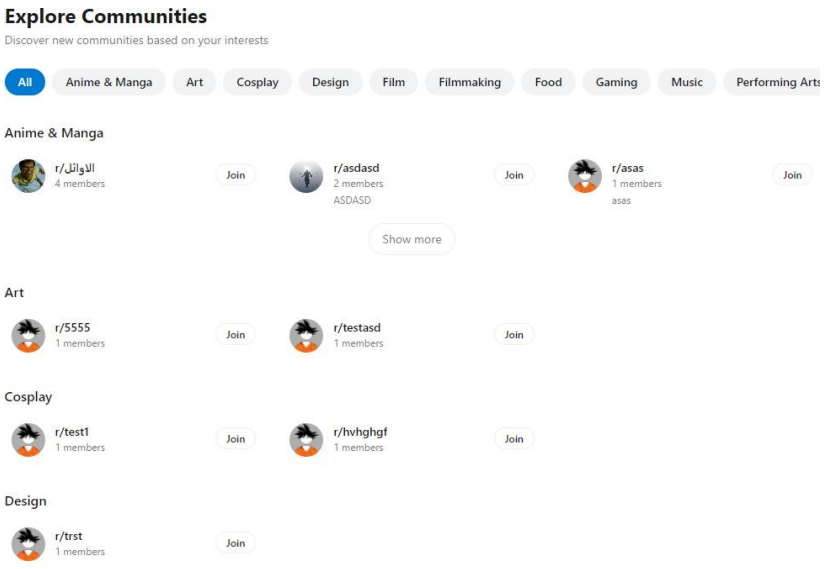


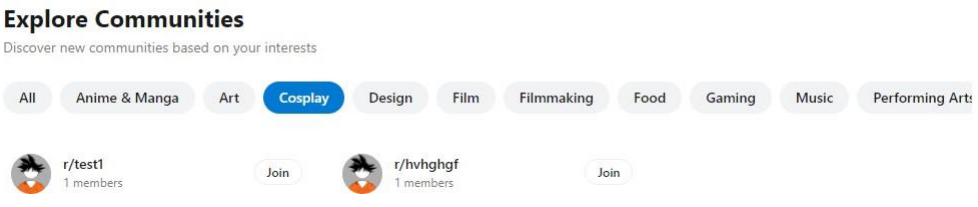
8.3 Creating a community

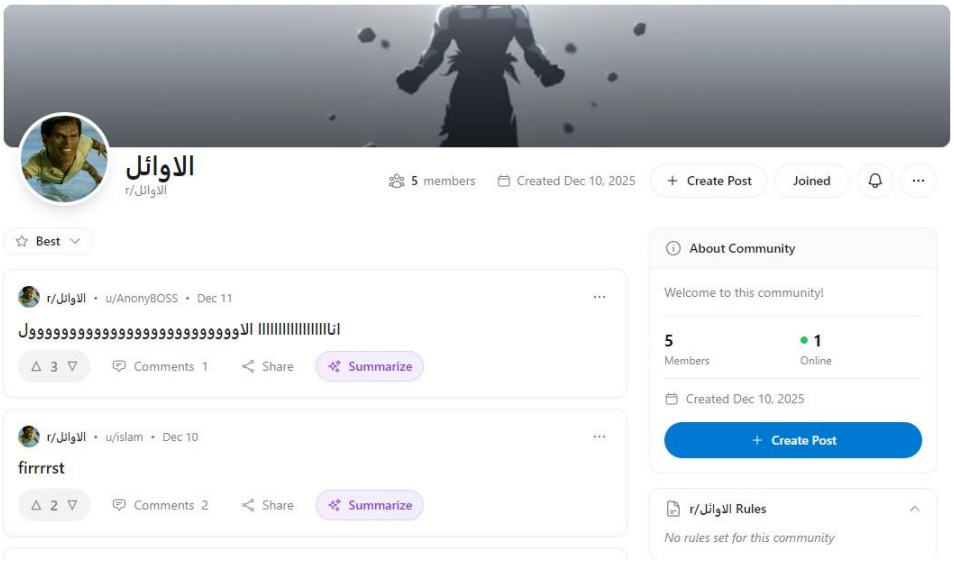




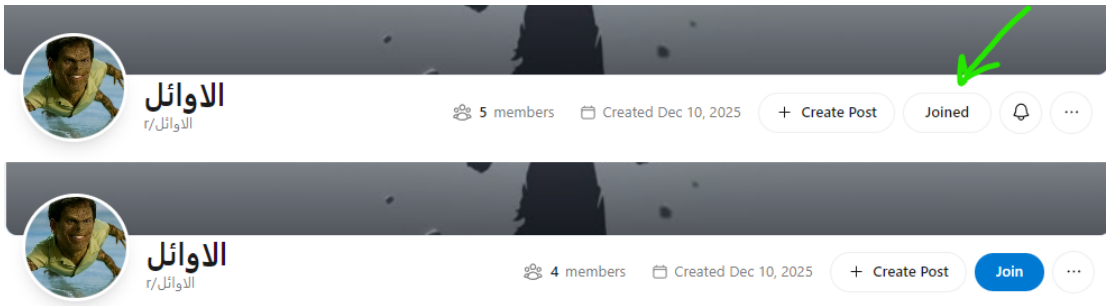
8.4 Joining/Leaving a community



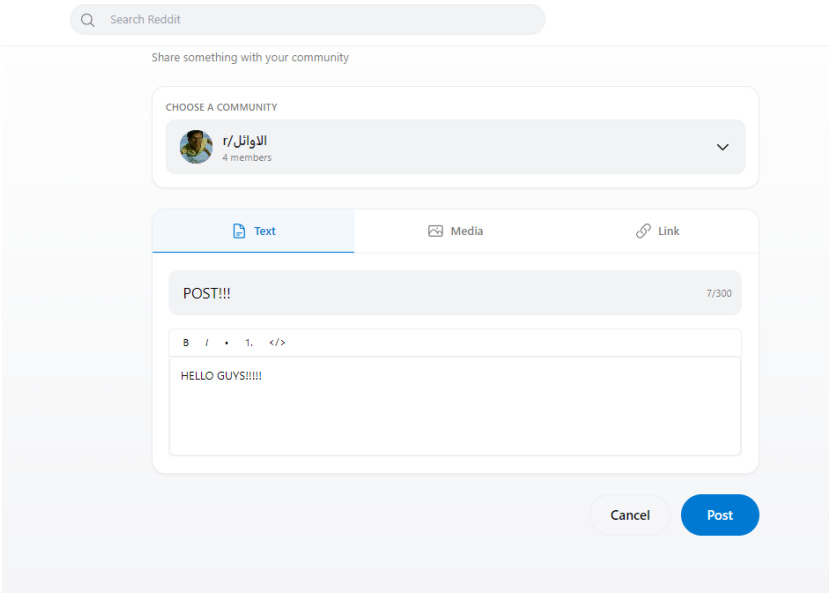


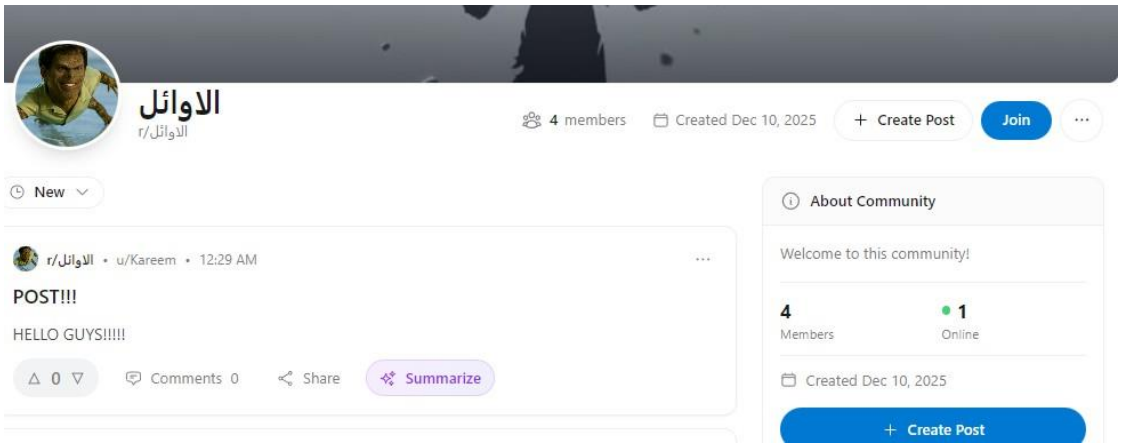


To leave a community, click the "Joined" button, and you will be removed from the community

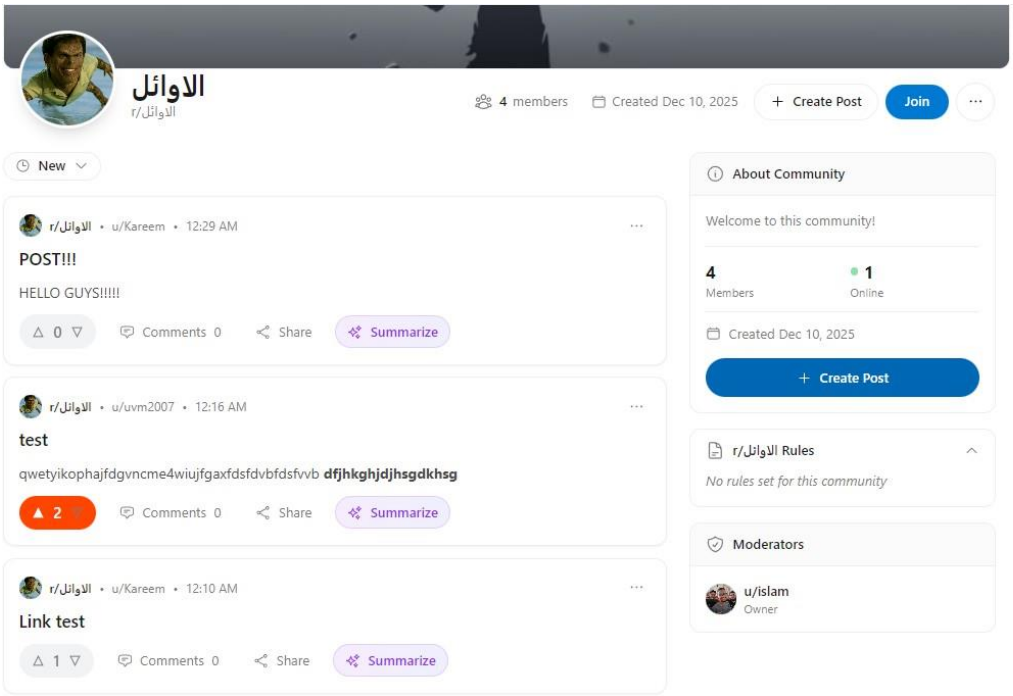


8.5 Creating a post in a community

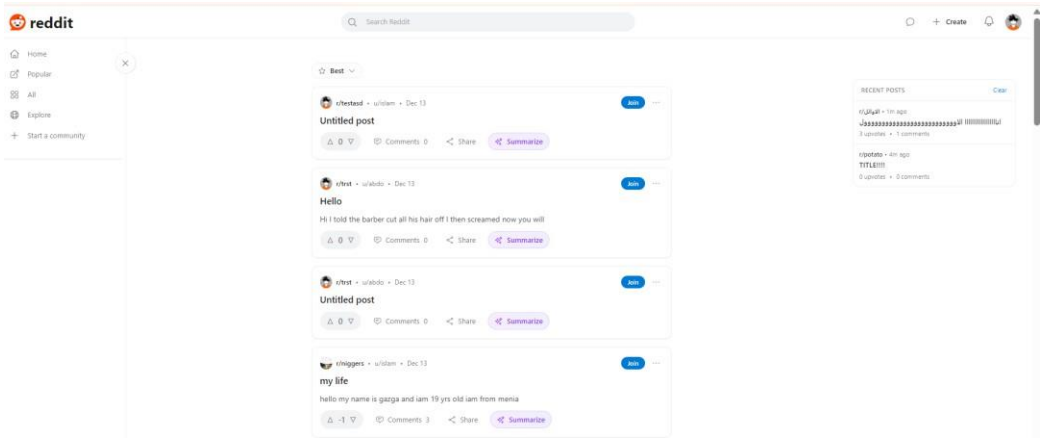




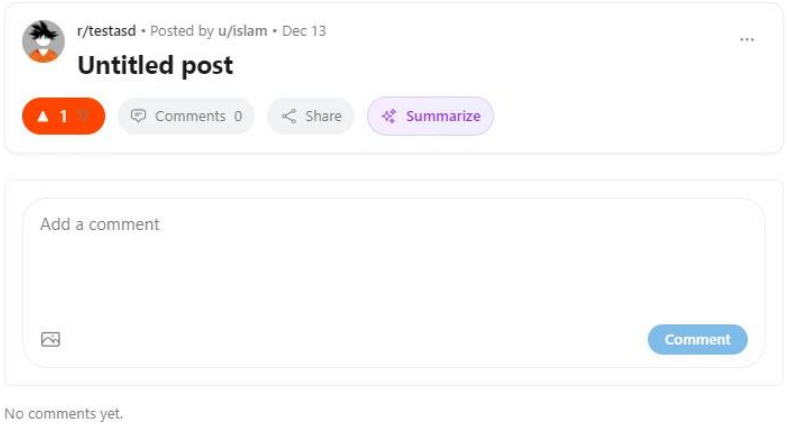
8.6 Viewing posts in a community

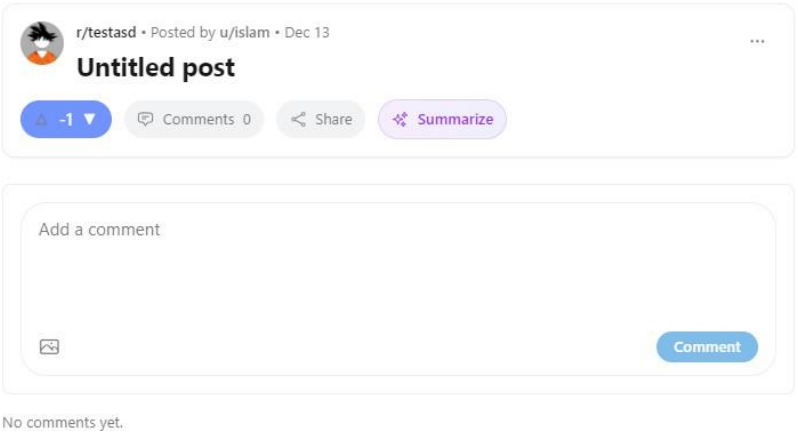


8.7 Viewing feed page

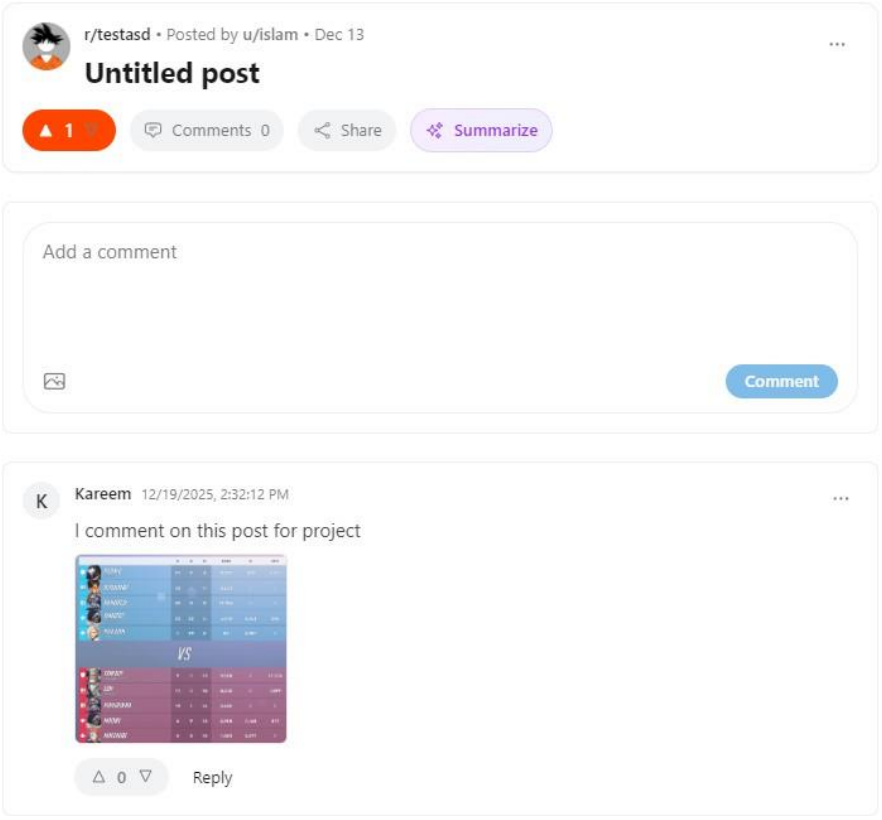


8.8 Upvoting/Downvoting posts

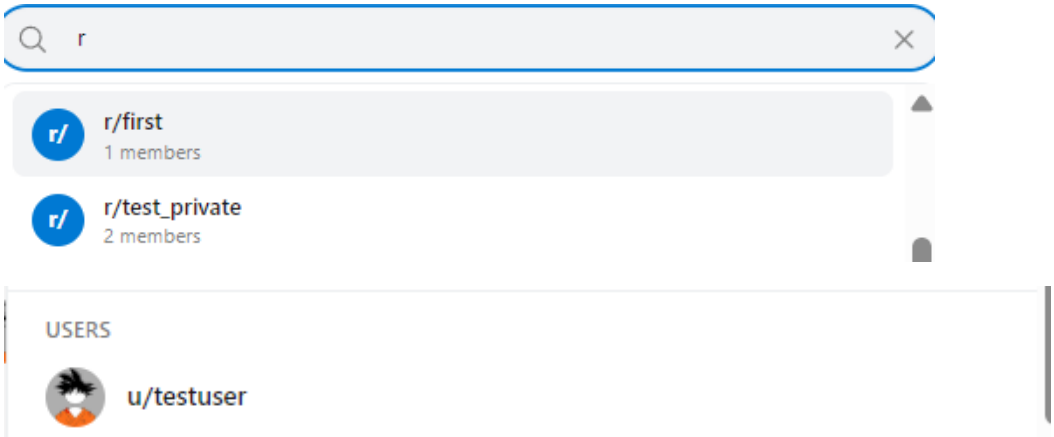




8.9 Commenting on posts



8.10 Searching for communities and users



POSTS

My first post

r/unknown • u/saif

My first post

r/unknown • u/saif

My first post

r/unknown • u/saif

My first post

r/unknown • u/saif

a

r/unknown • u/saif

9. Validation

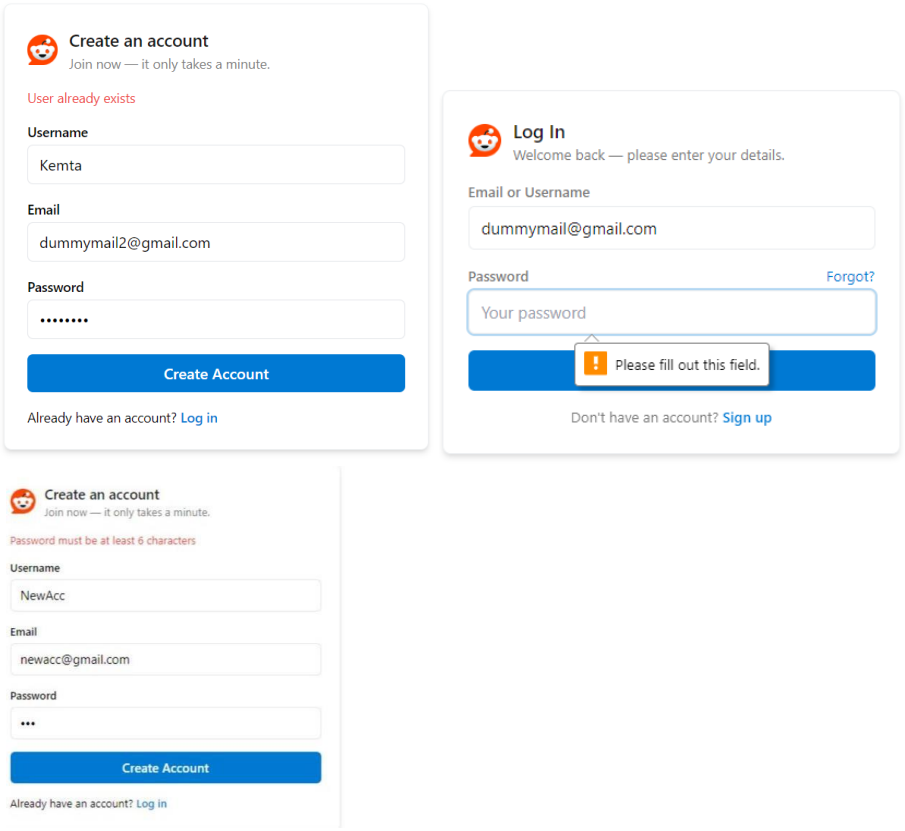
9.1 Schema-Level Validations

- Required fields are enforced on essential attributes such as usernames, emails, passwords, post titles, and comment bodies to prevent incomplete records.

- Unique constraints are applied to fields like usernames, email addresses, and community names to avoid duplication.

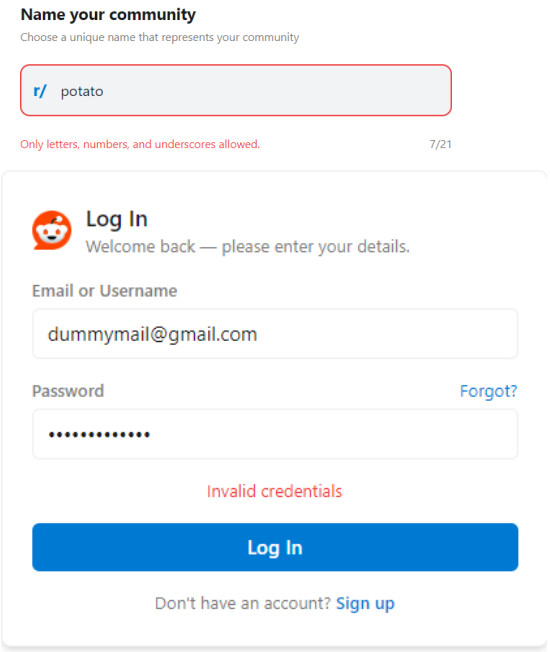
- Enumerated values are used for voting values, allowing only +1 or -1 to ensure valid voting behavior.

- Data type enforcement ensures fields follow the correct type (String, Number, Date), preventing inconsistent or malformed data.



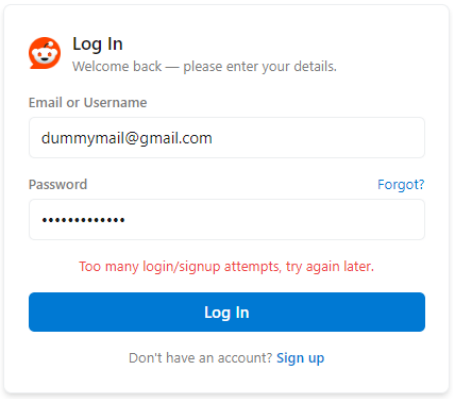
9.2 Route-Level Validations

- Missing field validation to ensure required request fields are provided.
- Duplicate record checks to prevent creating users or communities that already exist.
- Credential validation to detect incorrect login details and return appropriate error messages.
- ObjectId validation to ensure MongoDB IDs are valid before querying the database.
- Vote value validation to restrict votes to valid values only (+1 or -1).
- File upload validation, including file presence, maximum file size (5MB), and allowed file types.
- Self-action prevention, such as preventing users from following themselves.
- Search query validation to ensure search requests include a query parameter.
- Request duplication prevention, particularly for join requests to private communities.



9.3 Rate Limiting Validations

- Authentication rate limiter: Limits login and signup requests to 20 requests per 15 minutes per IP address.
- Write operation rate limiter: Limits create and update operations (such as posting and commenting) to 30 requests per minute.



9.4 Authentication and Authorization Validation

- JWT token verification is handled through authentication middleware, which validates the token before granting access to protected routes.
- User authorization checks ensure that only the owner of a resource (such as a post or comment) can edit or delete it.
- Access control prevents unauthorized users from performing restricted actions within communities

10. API docx

https://documenter.getpostman.com/view/47437297/2sB3dWqmfS

11. Github and Deployment Link

https://github.com/AbdulrhmanHammouda/reddit-clone/tree/main

https://reddit-clone-iota-taupe.vercel.app/

12. Team Contributions

- **Abdelrahman Wael: Account & Profile**
- **Ahmed Hesham: Communities**
- **Marwan Wagih: Posts**
- **Ahmed Hossam: Feed & Voting**
- **Kareem Ahmed: Comments**
- **Seif Amjad: Search, Messaging & AI**